

Mathematical model:

Derive Euler Lagrange equations of the system to find the position, velocity, acceleration and forces acting on the arm at a given time.

- First have a complete detailed diagram of the system.
- Define a coordinate system, Cartesian or angular.
- Analyse linear kinetic and potential energies.
- Use Euler-Lagrange to find system of equations.
- plug into MATLAB.

Design ideas:



Servo motors, electronics and coding:

- With consideration of the mathematical model, usage of PWM signals to control the RPM of the motors and the angle of the limbs of the arm.
- Raspberry pies offer 3.3V output pins and sometimes 5V pins depending on the pie used.
- If the motors require 5V and only 3.3V is available, an external power supply may be needed in between the output pins on the pie and motor input.
- Servos have 3 wires, PWM, power and ground.
- We'll need a pin layout of the raspberry pie we choose to use.
- SG90 servos require a 50Hz signal, if we need metal gears instead of plastic, MG90 could be a better choice for durability.
- A 50Hz signal has a PWM range of 2ms.
- When the PWM signal is high for 1ms, it moves the motor arm to 0 degrees, when high for 2ms it moves motor to 180 degrees, 1.5ms 90 degrees etc...
- Duty cycle must be described as a percentage in python, not in seconds, so a conversion is required to determine percent.
- A 50Hz signal has a total period of 20ms. So $100\% = 20\text{ms}$. Since we need a range of between 1-2 ms, 1 ms is one twentieth of 20 ms which corresponds to 5 % making 2ms 10 %.
- The sleep function in python, `time.sleep(time)` determines how long the function stop before running it again, it determines how fast the servos move.
- If sleep time is decreased, loop runs faster means faster servo movement etc. This loop can be broken when the input is cancelled or when another constraint is defined.
- Noise can effect PWM signals, exterior pin board can be connected to the raspberry pie to get rid of noise in the duty cycle if it shows up.
- Python libraries are required to control an exterior pin board if we use one.

Existing APIs for wii remote and controller:

- Translate wii remote to position in space,
<https://github.com/RaphaelWimmer/wiimote.py/blob/master/wiimote.py>
- Translate xbox controller to position in space,
<https://github.com/martinohanlon/XboxController/blob/master/XboxController.py>

-Translate 8-bit do controller to position in space,

https://github.com/kawamataryo/8bitdo_zero2

-I think the 8-bit do controller uses less code compared to the others.